

Python List (Module-6)

Lists are used to store multiple items in a single variable. Lists are created using square brackets:

Example

```
thislist = ["apple", "banana", "cherry"]  
print(thislist)
```

Empty List

```
x=[]  
Here x is an empty list.
```

Nested List

List inside another list is called nested list.

```
x=[10,20,[1,2,3,4]]  
Here x is a nested list.
```

Usage of list()

The list() function is used to convert an iterable (such as a string, tuple or dictionary) into a list. It takes an iterable as an argument and returns a list with each element of the iterable as an item in the list.

```
s="hello"  
print(list(s))
```

Output:

```
['h','e','l','l','o']
```

Access Items

List items are ordered, changeable, and allow duplicate values. List items are indexed, the first item has index [0], the second item has index [1] etc.

```
thislist = ["apple", "banana", "cherry"]  
print(thislist[1])
```

Output:

```
banana
```

Length of the string

len() is used to find out the length of the string.

```
x= [10, 20, 30]  
print(len(x))
```

Output:

```
3
```

Negative Indexing

Negative indexing means start from the end. -1 refers to the last item, -2 refers to the second last item etc.

```
thislist = ["apple", "banana", "cherry"]  
print(thislist[-1])
```

Output:

cherry

Range of Indices

You can specify a range of indices by specifying where to start and where to end the range. When specifying a range, the return value will be a new list with the specified items.

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]  
print(thislist[2:5])
```

Output:

['cherry', 'orange', 'kiwi']

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]  
print(thislist[:4])
```

Output:

['apple', 'banana', 'cherry', 'orange']

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]  
print(thislist[2:])
```

Output:

```
['cherry', 'orange', 'kiwi', 'melon', 'mango']  
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]  
print(thislist[-4:-1])
```

Output:

['orange', 'kiwi', 'melon']

Change Item Value

To change the value of a specific item, refer to the index number:

```
thislist = ["apple", "banana", "cherry"]  
thislist[1] = "blackcurrant"  
print(thislist)
```

Output:

['apple', 'blackcurrant', 'cherry']

Change a Range of Item Values

To change the value of items within a specific range, define a list with the new values, and refer to the range of index numbers where you want to insert the new values:

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]  
thislist[1:3] = ["blackcurrant", "watermelon"]  
print(thislist)
```

Output:

```
['apple', 'blackcurrant', 'watermelon', 'orange', 'kiwi', 'mango']
```

If you insert *more* items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly:

```
thislist = ["apple", "banana", "cherry"]  
thislist[1:2] = ["blackcurrant", "watermelon"]  
print(thislist)
```

Output:

```
['apple', 'blackcurrant', 'watermelon', 'cherry']
```

Note: The length of the list will change when the number of items inserted does not match the number of items replaced.

If you insert *less* items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly:

```
thislist = ["apple", "banana", "cherry"]  
thislist[1:3] = ["watermelon"]  
print(thislist)
```

Output:

```
['apple', 'watermelon']
```

Traversing a list

```
x= [10, 20, 30, 40, 50]  
for y in x:  
    print(y)
```

Output:

```
[10, 20, 30, 40, 50]
```

```
x= [10, 20, 30, 40, 50]  
for y in range(len(x)):  
    print(x[y])
```

Output:

```
[10, 20, 30, 40, 50]
```

Comparing lists

We can compare lists by using '=' operator.

```
x= [10, 20, 30, 40, 50]  
y=[1, 2, 3, 4, 5]  
print(x==y)
```

Output:

```
False
```

Joining lists

We can join two lists by using '+' operator.

```
x= [10, 20, 30, 40, 50]
```

```
y=[1, 2, 3, 4, 5]
```

```
z=x + y
```

```
print(z)
```

Output:

```
[10, 20, 30, 40, 50, 1, 2, 3, 4, 5]
```

Replication of list

We can replicate a list by using '*' operator.

```
x=[1, 2, 3, 4, 5]
```

```
y=x*2
```

```
print(y)
```

Output:

```
[1, 2, 3, 4, 5, 1, 2, 3, 4, 5]
```

List Methods

append()

The append() method appends an element to the end of the list.

Syntax

```
list.append(elmnt)
```

```
fruits = ["apple", "banana", "cherry"]
```

```
fruits.append("orange")
```

```
print(fruits)
```

Output:

```
['apple', 'banana', 'cherry', 'orange']
```

```
a = ["apple", "banana", "cherry"]
```

```
b = ["Ford", "BMW", "Volvo"]
```

```
a.append(b)
```

```
print(a)
```

Output:

```
['apple', 'banana', 'cherry', ['Ford', 'BMW', 'Volvo']]
```

clear()

The clear() method removes all the elements from a list.

Syntax

```
list.clear()
```

```
fruits = ["apple", "banana", "cherry"]
```

```
fruits.clear()
```

```
print(fruits)
```

Output:

```
[]
```

copy()

The copy() method returns a copy of the specified list.

Syntax

```
list.copy()
```

```
fruits = ['apple', 'banana', 'cherry', 'orange']
```

```
x = fruits.copy()
```

```
print(x)
```

Output:

```
['apple', 'banana', 'cherry']
```

count()

The count() method returns the number of elements with the specified value.

Syntax

```
list.count(value)
```

```
fruits = [1, 4, 2, 9, 7, 8, 9, 3, 1]
```

```
x = fruits.count(9)
```

```
print(x)
```

Output:

```
2
```

extend()

The extend() method adds the specified list elements (or any iterable) to the end of the current list.

Syntax

```
list.extend(iterable)
```

```
fruits = ['apple', 'banana', 'cherry']
```

```
points = [1, 4, 5, 9]
```

```
fruits.extend(points)
```

```
print(fruits)
```

Output:

```
['apple', 'banana', 'cherry', 1, 4, 5, 9]
```

index()

The index() method returns the position at the first occurrence of the specified value.

Syntax

```
list.index(elmnt)
```

```
fruits = [4, 55, 64, 32, 16, 32]  
x = fruits.index(32)  
print(x)
```

Output:

3

insert()

The insert() method inserts the specified value at the specified position.

Syntax

```
list.insert(pos, elmnt)
```

```
fruits = ['apple', 'banana', 'cherry']  
fruits.insert(1, "orange")  
print(fruits)
```

Output:

['apple', 'orange', 'banana', 'cherry']

pop()

The pop() method removes the element at the specified position and returns it.

Syntax

```
list.pop([pos])
```

Number specifying the position of the element you want to remove, default value is -1, which returns the last item.

```
fruits = ['apple', 'banana', 'cherry']  
x = fruits.pop(1)  
print(x)
```

Output:

banana

remove()

The remove() method removes the first occurrence of the element with the specified value.

Syntax

```
list.remove(elmnt)
```

```
fruits = ['apple', 'banana', 'cherry']  
fruits.remove("banana")  
print(fruits)
```

Output:

['apple', 'cherry']

reverse()

The reverse() method reverses the sorting order of the elements.

Syntax

```
list.reverse()
```

```
fruits = ['apple', 'banana', 'cherry']  
fruits.reverse()  
print(fruits)
```

Output:

```
['cherry', 'banana', 'apple']
```

sort()

The sort() method sorts the list ascending by default.

You can also make a function to decide the sorting criteria(s).

Syntax

```
list.sort([reverse=True|False], [key=myFunc])
```

reverse	Optional. reverse=True will sort the list descending. Default is reverse=False
key	Optional. A function to specify the sorting criteria(s)

```
cars = ['Ford', 'BMW', 'Volvo']  
cars.sort(reverse=True)  
print(cars)
```

Output:

```
['Volvo', 'Ford', 'BMW']
```

Sort the list by the length of the values:

A function that returns the length of the value:

```
def myFunc(e):  
    return len(e)
```

```
cars = ['Ford', 'Mitsubishi', 'BMW', 'VW']  
cars.sort(key=myFunc)  
print(cars)
```

Output:

```
['VW', 'BMW', 'Ford', 'Mitsubishi']
```

List Comprehension

List comprehension offers a shorter syntax when you want to create a new list based on the values of an existing list.

The Syntax

newlist = [expression for item in iterable if condition]

The return value is a new list, leaving the old list unchanged.

```
newlist = [x for x in range(1,11)]  
print(newlist)
```

Output:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
newlist = [x for x in range(1,11) if x%2==0]  
print(newlist)
```

Output:

```
[2, 4, 6, 8, 10]
```

Python zip() Function

The zip() function returns a zip object, which is an iterator of tuples where the first item in each passed iterator is paired together, and then the second item in each passed iterator are paired together etc.

If the passed iterables have different lengths, the iterable with the least items decides the length of the new iterator.

zip(iterator1, iterator2, iterator3 ...)

```
a = ("John", "Charles", "Mike")  
b = ("Jenny", "Christy", "Monica", "Vicky")  
x = list(zip(a, b))  
print(x)
```

Output:

```
[('John', 'Jenny'), ('Charles', 'Christy'), ('Mike', 'Monica')]
```